



09

가상 함수와 추상 클래스

학습 목표

1. 상속에서 함수 재정의を理解한다.
2. 가상 함수와 오버라이딩, 동적바인딩의 개념을 이해한다.
3. 가상 소멸자의 중요성을 이해한다.
4. 가상 함수를 활용하여 프로그램을 작성할 수 있다.
5. 순수가상함수와 추상 클래스를 이해하고 작성할 수 있다.

예제 9-1 파생클래스에서 함수 재정의

3

```
#include <iostream>
using namespace std;
```

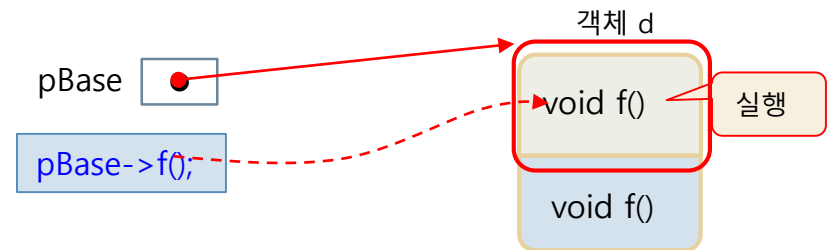
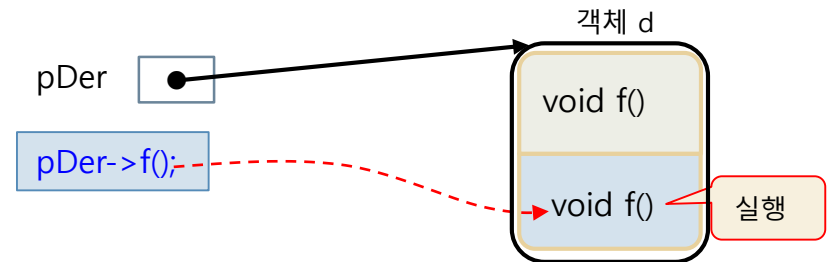
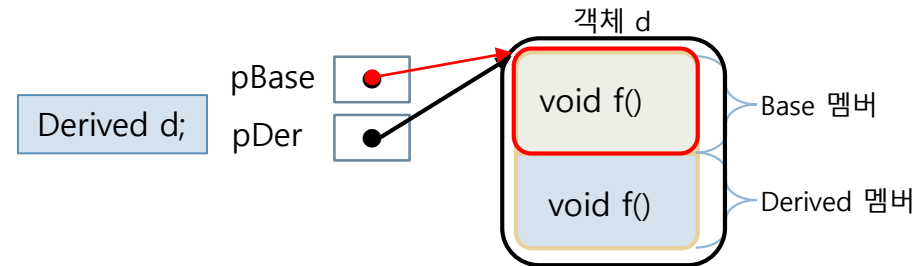
```
class Base {
public:
    void f() { cout << "Base::f() called" << endl; }
};
```

```
class Derived : public Base {
public:
    void f() { cout << "Derived::f() called" << endl; }
};
```

```
void main() {
    Derived d, *pDer;
    pDer = &d;
    pDer->f(); // Derived::f() 호출
}
```

```
Base* pBase;
pBase = pDer; // 업캐스팅
pBase->f(); // Base::f() 호출
}
```

```
Derived::f() called
Base::f() called
```



가상 함수와 오버라이딩

4

□ 가상 함수(virtual function)

- virtual 키워드로 선언된 멤버 함수
- virtual 키워드의 의미

- 동적 바인딩(실행시간에 호출할 함수를 결정하는 것) 지시어
- 컴파일러에게 함수에 대한 호출 바인딩을 실행 시간까지 미루도록 지시

```
class Base {  
public:  
    virtual void f() {  
        cout << "Base::f() called" << endl;  
    }  
};
```

가상 함수

□ 함수 오버라이딩(function overriding)

- 파생 클래스에서 기본 클래스의 가상 함수와 동일한 이름의 함수 선언
 - 기본 클래스의 가상 함수의 존재감 상실시킴
 - 파생 클래스에서 오버라이딩한 함수가 호출되도록 동적 바인딩
 - 가상 함수의 재정의라고도 부름
 - 다형성의 한 종류

```
class Derived : public Base {  
public:  
    void f() override {  
        cout << "Derived::f() called" << endl;  
    }  
};
```

가상 함수 재정의
즉, 오버라이딩

함수 재정의와 오버라이딩 사례 비교

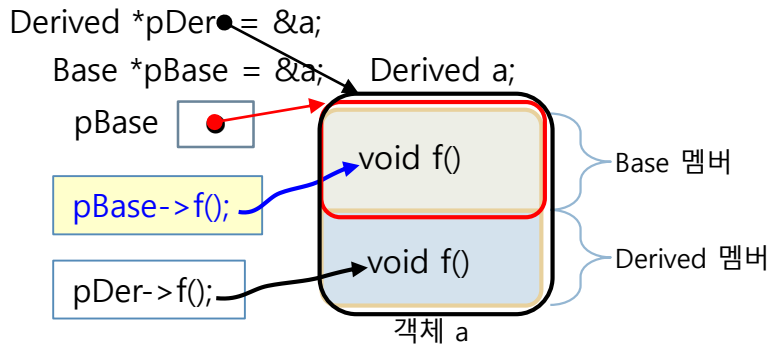
5

```
class Base {
public:
    void f() {
        cout << "Base::f() called" << endl;
    }
};

class Derived : public Base {
public:
    void f() {
        cout << "Derived::f() called" << endl;
    }
};
```

함수 재정의

함수 재정의(redefine)



(a) a 객체에는 동등한 호출 기회를 가진 함수 f()가 두 개 존재

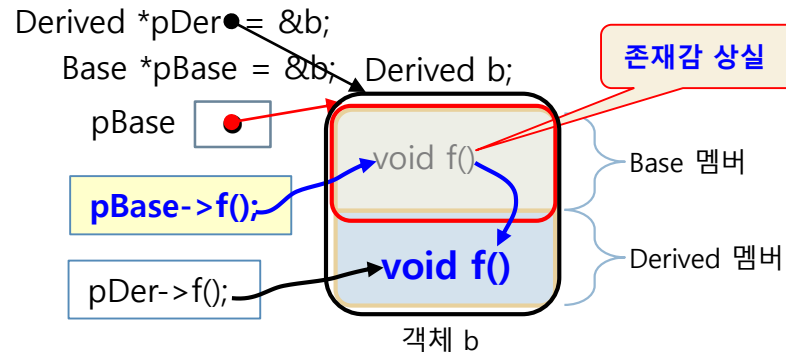
```
class Base {
public:
    virtual void f() {
        cout << "Base::f() called" << endl;
    }
};

class Derived : public Base {
public:
    void f() override {
        cout << "Derived::f() called" << endl;
    }
};
```

가상 함수

가상 함수 재정의 즉, 오버라이딩

오버라이딩(overriding)



(b) b 객체에는 두 개의 함수 f()가 존재하지만, Base의 f()는 존재감을 잃고, 항상 Derived의 f()가 호출됨

정적 바인딩과 동적 바인딩

6

```
class Base {
public:
    void f() {
        cout << "Base::f() called" << endl;
    }
};

class Derived : public Base {
public:
    void f() {
        cout << "Derived::f() called" << endl;
    }
};
```

함수 재정의

정적 바인딩(static binding)

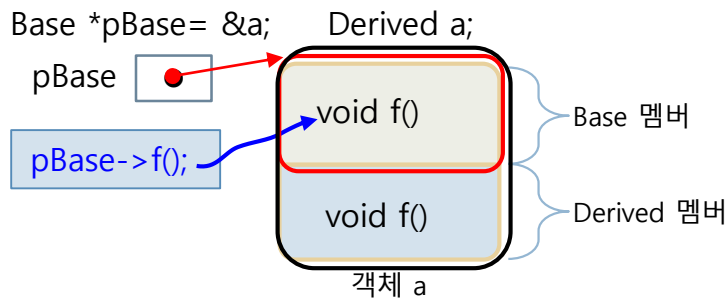
```
class Base {
public:
    virtual void f() {
        cout << "Base::f() called" << endl;
    }
};

class Derived : public Base {
public:
    void f() override {
        cout << "Derived::f() called" << endl;
    }
};
```

가상 함수

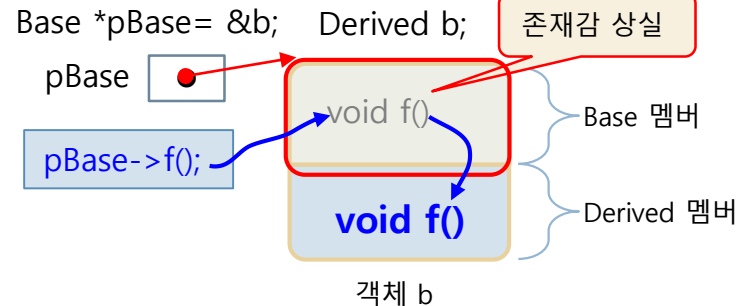
가상 함수 재정의
즉, 오버라이딩

동적 바인딩(dynamic binding)



(a) 정적 바인딩

컴파일 시간에 호출할 함수가 이미 결정됨



(b) 동적 바인딩

실행 시간에 호출할 함수가 결정됨

함수 재정의와 오버라이딩 용어의 혼란 정리

7

함수 재정의라는 용어를 사용할 때 신중을 기해야 한다. 가상 함수를 재정의하는 경우와 아닌 경우에 따라 프로그램의 실행이 완전히 달라지기 때문이다([그림 9-3] 참고).

가상 함수를 재정의하는 **오버라이딩**의 경우, 호출할 함수를 실행 시간에 결정하는 **동적 바인딩**이 일어나지만, 그렇지 않은 경우 컴파일 시간에 결정된 함수가 단순히 호출된다 (정적 바인딩).

저자는 **가상 함수를 재정의하는 것을 오버라이딩**으로, 그렇지 않는 경우를 **함수 재정의**로 구분하고자 한다.

Java의 경우 이런 혼란은 없다. 멤버 함수가 가상이나 아니냐로 구분되지 않으며, 함수 재정의는 곧 오버라이딩이며, **무조건 동적 바인딩**이 일어난다.

예제 9-2 오버라이딩과 가상 함수 호출

8

```
#include <iostream>
using namespace std;

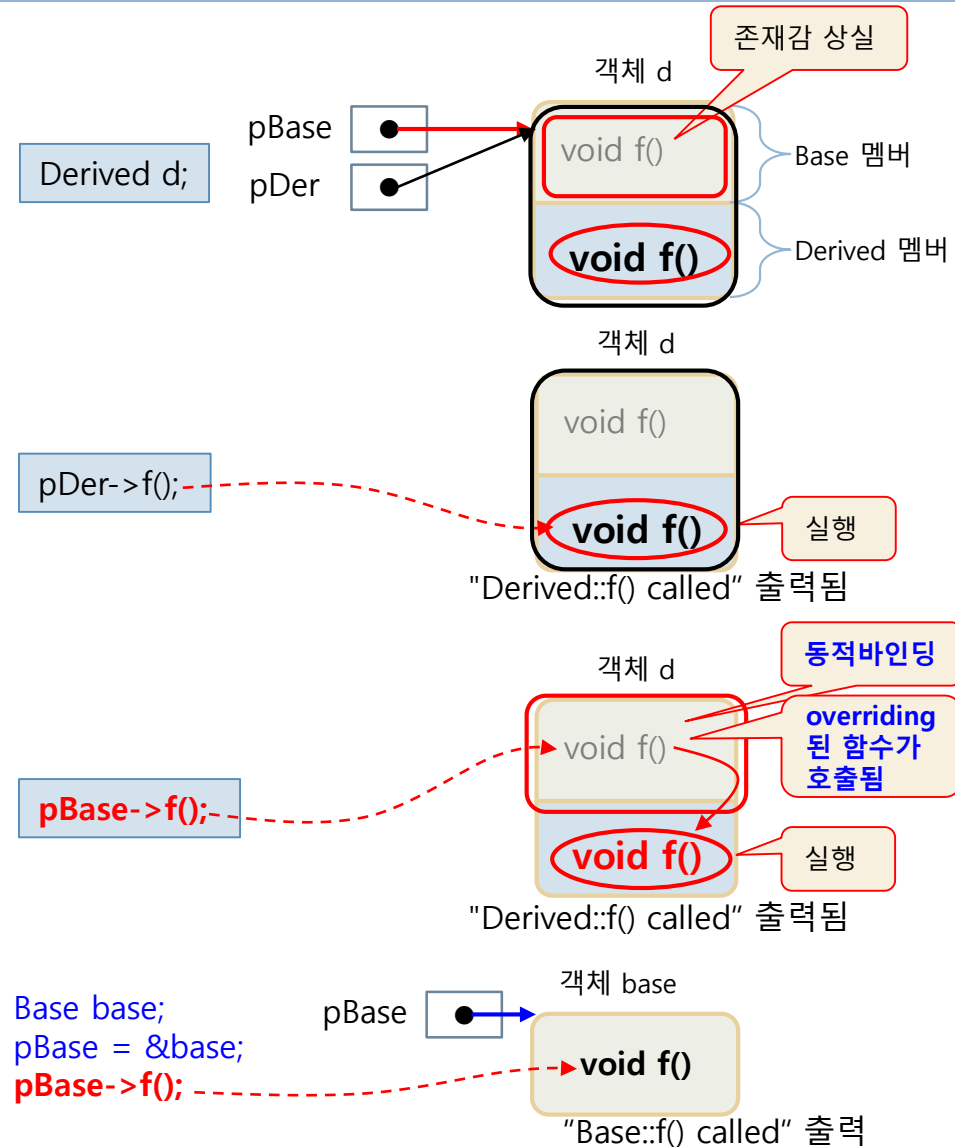
class Base {
public:
    virtual void f() { cout << "Base::f() called" << endl; }
};

class Derived : public Base {
public:
    void f() override { cout << "Derived::f() called" << endl; }
};

int main() {
    Derived d, *pDer; // 파생클래스의 객체와 포인터
    pDer = &d;
    pDer->f(); // Derived::f() 호출

    Base * pBase; // 기본클래스의 포인터
    pBase = pDer; // 업 캐스팅
    pBase->f(); // 동적 바인딩 발생!! Derived::f() 실행
}
```

Derived::f() called
Derived::f() called



동적 바인딩

9

```
class Base {  
public:  
    virtual void f() {  
        cout << "Base::f() called" << endl;  
    }  
};
```

```
void callBase(Base *pb) {  
    pb->f();  
};
```

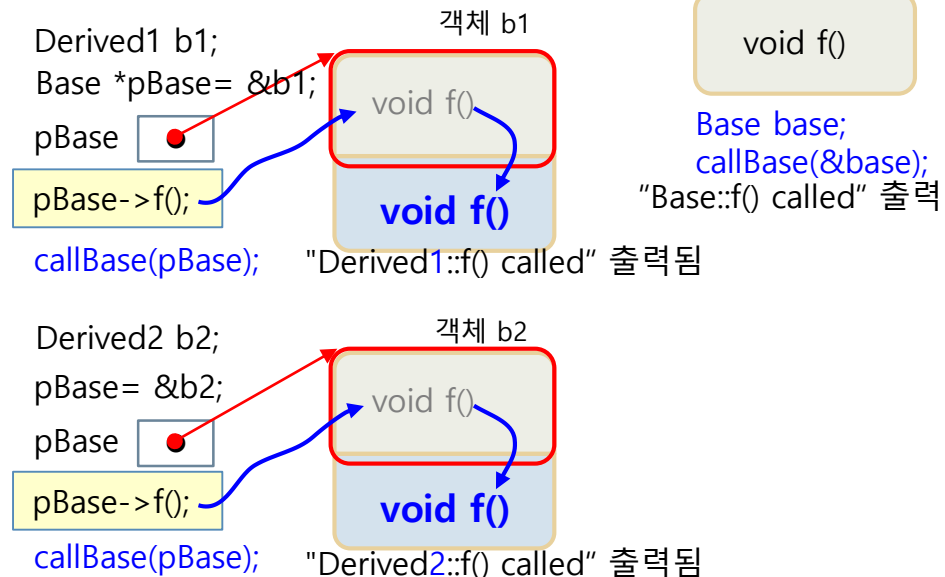
pb



- 동적 바인딩(호출될 함수가 실행 시간에 결정됨)
 - 가상 함수 호출 시 오버라이딩된 함수가 무조건 호출
 - 파생 클래스의 객체에 대해 이를 가리키는
 - 기본 클래스에 대한 포인터를 이용하여 가상 함수를 호출했을 때
 - 객체 내에 오버라이딩된 파생 클래스의 함수를 찾아 실행함
 - 호출할 함수를 실행 시간에 결정하여 해당 오버라이딩 함수를 실행
 - 실행시간 바인딩, 런타임 바인딩, 늦은 바인딩으로 불림

```
class Derived1 : public Base {  
public:  
    void f() override {  
        cout << "Derived1::f() called" << endl;  
    }  
};
```

```
class Derived2 : public Base {  
public:  
    void f() override {  
        cout << "Derived2::f() called" << endl;  
    }  
};
```



오버라이딩의 목적 - 다형성의 실현

10

오버라이딩을 통한 다형성의 실현

- draw() 가상 함수를 가진 기본 클래스 Shape
- 오버라이딩을 통해 Circle, Rect, Line 클래스에서 자신만의 draw() 구현

```
Circle *pCircle = new Circle();  
pCircle->draw(); // Circle을 그린다.  
  
Rect *pRect = new Rect();  
pRect->draw(); // Rect을 그린다.  
  
Line *pLine = new Line();  
pLine->draw(); // Line을 그린다.
```

```
class Shape {  
public:  
    virtual void draw() { }  
};
```

가상 함수 선언.

파생 클래스에서 재정의할 함수에 대한 인터페이스 역할

```
class Circle : public Shape {  
public:  
    void draw() override {  
        // Circle을 그린다.  
    }  
};
```

```
class Rect : public Shape {  
public:  
    void draw() override {  
        // Rect을 그린다.  
    }  
};
```

```
class Line : public Shape {  
public:  
    void draw() override {  
        // Line을 그린다.  
    }  
};
```

오버라이딩,
다형성 실현

p

```
void paint(Shape* p) {  
    p->draw();  
}
```

```
paint(new Circle()); // Circle을 그린다.  
paint(new Rect()); // Rect을 그린다.  
paint(new Line()); // Line을 그린다.
```

p가 가리키는 객체에 오버라이딩된 draw() 호출.
여기서 비록 동일한 함수를 호출하지만 서로 다른 도형 객체에 대해 오버라이딩된 서로 다른 함수가 호출됨
오버라이딩을 통한 다형성의 실현
다형성: 하나의 인터페이스에 대해 서로 다른 모양의 구현

Circle 객체

Rect 객체

Line 객체

```
void draw()  
void draw()
```

```
void draw()  
void draw()
```

```
void draw()  
void draw()
```

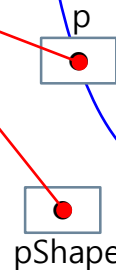
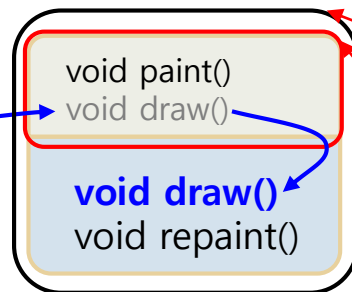
동적 바인딩이 발생하는 구체적 경우

11

- ① 기본 클래스 내의 멤버 함수가 가상 함수 호출
- ② 파생 클래스 내의 멤버 함수가 가상 함수 호출
- ③ main()과 같은 외부 함수에서 기본 클래스의 포인터로 가상 함수 호출
- ④ 다른 클래스에서 기본 클래스의 포인터로 가상 함수 호출

동적으로 할당된
Circle 객체

pShape->draw();



```
#include <iostream>
using namespace std;

class Shape {
public:
    void paint() { // ①
        draw();
    }
    virtual void draw() {
        cout << "Shape::draw() called" << endl;
    };
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Circle::draw() called" << endl;
    }
    void repaint() { // ②
        draw();
    };
};

class Manager {
public:
    void display(Shape *p) {
        p->draw(); // ④
    };
};

int main() {
    Shape *pShape = new Circle(); // 업캐스팅
    pShape->draw(); // ③
    pShape->paint(); // ①
    Manager m;
    m.display(pShape); // ④
    ((Circle *)pShape)->repaint(); // ②
    delete pShape;
}
```

동적바인딩

override 키워드의 사용 이점

12

```
class Shape {
public:
    virtual void draw(); // 가상함수
};

class Rect : public Shape {
public:
    void draw(); // 악!!

    // Shape의 draw()를 오버라이딩하려고 시도하였지만
    // 실수로 이름이 위처럼 틀린 경우,
    // 컴파일러는 Rect에 새로운 멤버 함수
    // draw()를 작성하려는 것으로 알고
    // 컴파일 오류를 발생시키지 않음
};
```

```
int main() {
    Rect r;
    r.draw(); // Shape의 draw()를 호출함
              // 원래는 Rect의 draw()를 호출할 의도였음
              // 그러나 에러 발생하지 않음
}
```

```
class Shape {
public:
    virtual void draw(); // 가상함수
};

class Rect : public Shape {
public:
    void draw() override; // 컴파일 오류 발생함

    // override는 컴파일러에게 오버라이딩을 확인하도록
    // 지시하는 것으로, 기본 클래스에 draw()라는 가상
    // 함수가 없으면 에러로 처리함
    // 개발자는 draw()의 작성 실수를 금방 파악 가능
};
```

final 키워드: 오버라이딩 금지 또는 상속 금지

13

```
class Shape {
public:
    virtual void draw(); // 가상함수
};

class Ellipse : public Shape {
public:
    // Shape의 draw()를 오버라이딩
    void draw() override final;
    // 추후 이 클래스를 상속 받아 오버라이딩 금지
};

class Circle : public Ellipse {
public:
    void draw() override;
    // 컴파일 오류: 금지된 오버라이딩 시도
};
```

```
class Shape {
public:
    virtual void draw(); // 가상함수
};

class Ellipse final : public Shape {
    // 추후 Ellipse 클래스의 상속을 금지시킴
public:
    void draw() override;
};

class Circle : public Ellipse {
    // 컴파일 오류: Ellipse 클래스는 더 이상 상속 안됨
public:
    ...
};
```

오버라이딩된 함수를 호출하는 동적 바인딩

```
#include <iostream>
using namespace std;
```

```
class Shape {
public:
```

```
void paint() {
draw();
}
```

```
virtual void draw() {
```

```
cout << "Shape::draw() called" << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
Shape *pShape = new Shape();
```

```
pShape->paint();
```

```
delete pShape;
```

```
}
```

기본 클래스의 객체에 대한 가상 함수 호출은 파생 클래스가 없기 때문에 자신의 함수가 호출됨
동적 바인딩이 일어나지 않음

Shape::draw() called

```
#include <iostream>
using namespace std;
```

```
class Shape {
public:
```

```
void paint() {
draw();
}
```

```
virtual void draw() {
```

```
cout << "Shape::draw() called" << endl;
```

```
}
```

```
};
```

```
class Circle : public Shape {
public:
```

```
void draw() override {
```

```
cout << "Circle::draw() called" << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
Shape *pShape = new Circle(); // 업캐스팅
```

```
pShape->paint();
```

```
delete pShape;
```

```
}
```

기본 클래스의 멤버 함수에서 가상 함수 호출은 파생 클래스(Circle)의 오버라이딩된 draw() 함수가 호출됨;
동적 바인딩

기본 클래스의 포인터를 이용한 paint() 호출은 궁극적으로 Circle에서 오버라이딩한 draw()를 호출함

Circle::draw() called

```
pShape = new Shape();
pShape->paint();
```

pShape

```
void paint()
void draw()
```

Shape
멤버

new Shape()

```
pShape = new Circle();
pShape->paint();
```

pShape

```
void paint()
void draw()
```

동적바인딩

Shape
멤버

Circle
멤버

```
void draw()
```

new Circle()

C++ 오버라이딩의 특징

15

▣ 오버라이딩의 성공 조건

- 가상 함수 이름, 매개 변수 타입과 개수, 리턴 타입이 모두 일치해야 함

```
class Base {  
public:  
    virtual void fail();  
    virtual void success();  
    virtual void g(int);  
};  
  
class Derived : public Base {  
public:  
    virtual int fail(); // 오버라이딩 실패. 리턴 타입이 다름  
    virtual void success(); // 오버라이딩 성공  
    virtual void g(int, double); // 오버로딩 사례. 정상 컴파일  
                                // 새로운 가상 함수 선언  
};
```

```
class Base {  
public:  
    virtual void f();  
};  
  
class Derived : public Base {  
public:  
    // overriding을 위한 모두 동일한 선언  
    virtual void f();  
    void f(); // virtual 생략 가능  
    void f() override; // 명확성을 위해 override 추가  
};
```

▣ 오버라이딩 시 virtual 지시어 생략 가능

- 가상 함수의 virtual 지시어는 상속됨, 파생 클래스에서 virtual 생략 가능

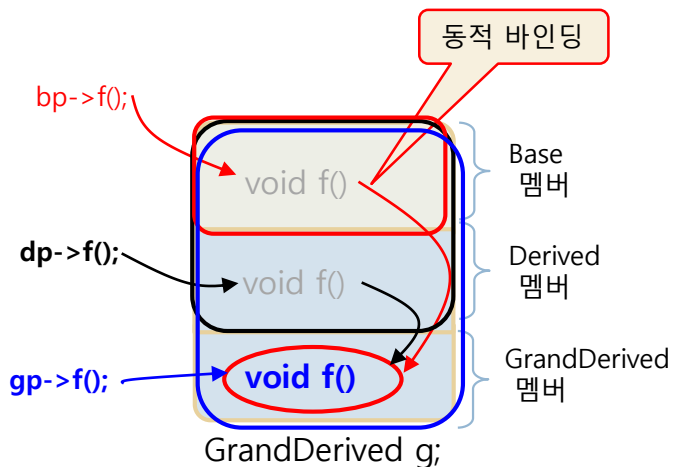
▣ 가상 함수의 접근 지정

- private, protected, public 중 자유롭게 지정 가능

예제 9-3 상속이 반복되는 경우의 가상 함수 호출

16

Base, Derived, GrandDerived가 상속 관계에 있을 때, 다음 코드를 실행한 결과는 무엇인가?



GrandDerived::f() called
GrandDerived::f() called
GrandDerived::f() called

```
class Base {
public:
    virtual void f() { cout << "Base::f() called" << endl; }
};

class Derived : public Base {
public:
    void f() override { cout << "Derived::f() called" << endl; }
};

class GrandDerived : public Derived {
public:
    void f() override { cout << "GrandDerived::f() called" << endl; }
};

int main() {
    GrandDerived g;
    Base *bp;
    Derived *dp;
    GrandDerived *gp;

    bp = dp = gp = &g; // 업캐스팅

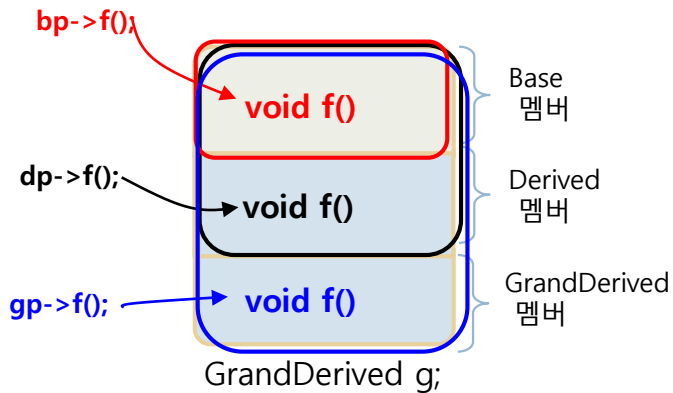
    bp->f();
    dp->f();
    gp->f();
}
```

동적 바인딩에 의해 모두
GrandDerived의 함수 f()
호출

상속이 반복되는 경우의 재정의 함수 호출

17

Base, Derived, GrandDerived가
상속 관계에 있을 때, 다음 코드를
실행한 결과는 무엇인가?



Base::f() called
Derived::f() called
GrandDerived::f() called

```
class Base {  
public:  
    void f() { cout << "Base::f() called" << endl; }  
};  
  
class Derived : public Base {  
public:  
    void f() { cout << "Derived::f() called" << endl; }  
};  
  
class GrandDerived : public Derived {  
public:  
    void f() { cout << "GrandDerived::f() called" << endl; }  
};  
  
int main() {  
    GrandDerived g;  
    Base *bp;  
    Derived *dp;  
    GrandDerived *gp;  
  
    bp = dp = gp = &g; // 업캐스팅  
  
    bp->f();  
    dp->f();  
    gp->f();  
}
```

가상 함수가 아니므로
정적 바인딩에 의해 각
클래스의 함수 f() 호출

오버라이딩과 범위 지정 연산자(::)

18

□ 범위 지정 연산자(::)

- ▣ 정적 바인딩 지시

- ▣ 기본클래스::가상함수() 형태로 기본 클래스의 가상 함수를 정적 바인딩으로 호출

- Shape::draw();

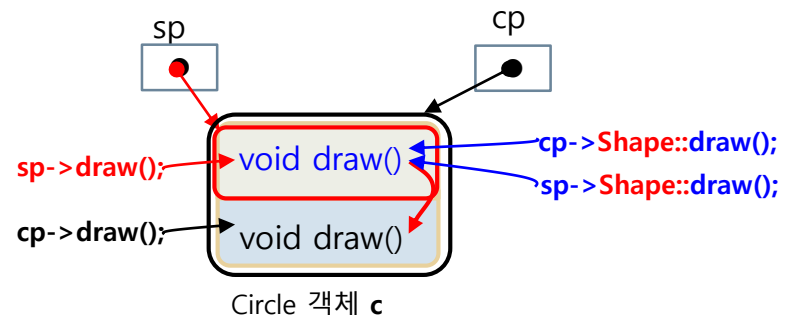
```
class Shape {  
public:  
    virtual void draw() {  
        cout << "Shape::draw() called" << endl;  
    }  
};
```

```
class Circle : public Shape {  
public:  
    void draw() override {  
        Shape::draw(); // 기본 클래스의 draw()를 실행한다.  
        cout << " Circle ::draw() called" << endl;  
    }  
};
```

실제 응용에서 많이 활용

```
Circle c, *cp = &c;  
cp->draw();           // Circle의 draw() 호출  
cp->Shape::draw();    // Shape의 draw() 호출
```

```
Shape *sp = cp;       // Up casting  
sp->draw();           // Circle의 draw() 호출  
sp->Shape::draw();    // Shape의 draw() 호출
```



예제 9-4 범위 지정 연산자(::)를 이용한 기본 클래스의 가상 함수 호출

19

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() {
        cout << "--Shape--";
    }
};

class Circle : public Shape {
public:
    void draw() override {
        Shape::draw(); // 기본 클래스의 draw() 호출
        cout << "Circle" << endl;
    }
};

int main() {
    Circle circle;
    Shape * pShape = &circle;

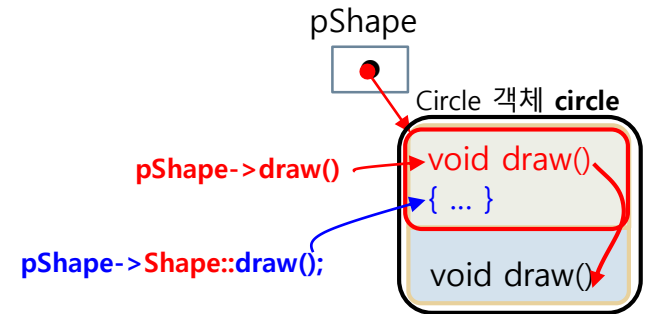
    pShape->draw();
    pShape->Shape::draw();
}
```

정적바인딩

동적바인딩

정적바인딩

실제 응용에서 많이 활용



--Shape--Circle
--Shape--

가상 소멸자

20

가상 소멸자

- 소멸자를 virtual 키워드로 선언
- 소멸자 호출 시 동적 바인딩 발생

```
class Base {  
public:  
    ~Base();  
};  
  
class Derived: public Base {  
public:  
    ~Derived();  
};
```

❶

~Base() 소멸자만 실행

```
int main() {  
    Base *p = new Derived();  
    delete p;  
}
```

❶ ~Base() 소멸자만 실행

소멸자가 가상 함수가 아닌 경우

```
int main() {  
    Derived *p = new Derived();  
    delete p;  
}
```

❶ ~Derived() 실행

❷ ~Base() 실행

소멸자가 가상 함수가 아닌 경우에도 정상 실행

```
class Base {  
public:  
    virtual ~Base();  
};  
  
class Derived: public Base {  
public:  
    virtual ~Derived() override;  
};
```

동적 바인딩

❶

```
int main() {  
    Base *p = new Derived();  
    delete p;  
}
```

❶ ~Base() 소멸자 호출

❷ ~Derived() 실행

❸ ~Base() 실행

가상 소멸자 경우

예제 9-6 소멸자를 가상 함수로 선언

21

- 기본 클래스의 소멸자는 항상 **virtual**로 선언할 것
 - ▣ 객체를 기본 클래스의 포인터로 소멸하든 파생 클래스의 포인터로 소멸하든, 항상 정상적인 소멸 과정이 진행되기 때문임

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual ~Base() { cout << "~Base()" << endl; }
};

class Derived: public Base {
public:
    ~Derived() override { cout << "~Derived()" << endl; }
};

int main() {
    Derived *dp = new Derived();
    Base *bp = new Derived();

    delete dp; // Derived의 포인터로 소멸
    delete bp; // Base의 포인터로 소멸
}
```

```
~Derived()
~Base()
~Derived()
~Base()
```

delete dp;

delete bp;

오버라이딩, 함수 재정의, 오버로딩

22

```
class Base {
public:
    void f() { cout << "Base::f() called" << endl; }
    virtual void rd() { cout << "Base::rd() called" << endl; }
};

class Derived : public Base {
public:
    // 함수 재정의: 상속관계(함수이름, 리턴, 인자 모두 동일)
    void f() { cout << "Derived::f() called" << endl; }

    // 오버라이딩: 상속관계(함수이름, 리턴, 인자 모두 동일)
    void rd() override { cout << "Derived::rd() called" << endl; }

    // 오버로딩: 클래스 내에서 함수 이름 동일, 인자 다름
    void f(int x)
    { cout << "Derived::f(int x) called" << endl; }

    void f(int x, int y)
    { cout << "Derived::f(int x, int y) called" << endl; }
};
```

```
void main() {
    Derived d, *pDer = &d;
    pDer->f();
    pDer->rd();
    pDer->f(1);
    pDer->f(1, 2);

    Base* pBase = pDer; // 업캐스팅
    pBase->f(); // Base::f() 호출
    pBase->rd(); // Derived::rd() 호출
}
```

```
Derived::f() called
Derived::rd() called
Derived::f(int x) called
Derived::f(int x, int y) called
Base::f() called
Derived::rd() called
```

함수 숨김(hidden)

23

- 파생 클래스의 함수는 기본 클래스의 동일한 이름의 함수 또는 전역 함수를 숨긴다.

```
void f() { cout << "f() called" << endl; } // 전역함수

class Base {
public:
    void f() { cout << "Base::f() called" << endl; }
};

class Derived : public Base {
public:
    void f(int x) {
        cout << "Derived::f() called" << x << endl;
        f(); // Compile Error(자신을 호출, 인자 없음)
        Base::f(); // OK
        ::f(); // OK. 전역함수
    }
};

void main() {
    Derived d, *pDer = &d;
    pDer->f(); // Base::f() 호출?, No, Compile Error
    pDer->f(3); // OK
    pDer->Base::f(); // OK
    f(); // OK, 전역함수
    Base* pBase = pDer; // 업캐스팅
    pBase->f(); // Base::f() 호출
}
```

```
void f() { cout << "f() called" << endl; } // 전역함수

class Base {
public:
    void f(int x) {
        cout << "Derived::f() called" << x << endl;
    }
};

class Derived : public Base {
public:
    void f(double x) {
        cout << "Derived::f() called" << x << endl;
        f(); // Compile Error
    }
};

void main() {
    Derived d, *pDer;
    pDer = &d;
    pDer->f(3); // Base::f(3) 호출?, No
                // Derived::f((double)3) 호출함
    Base* pBase;
    pBase = pDer; // 업캐스팅
    pBase->f(3); // Base::f(3) 호출
}
```

함수 중복(overloading)

24

- 같은 클래스 내의 함수들 또는 전역 함수들 끼리 함수 중복이 성립함

```
void f() { cout << "f() called" << endl; } // 전역함수
void f(double d) { cout << "Derived::f(d)\n"; }

class Base {
public:
    void f() { cout << "Base::f()" << endl; }
    void f(int x, char c) { cout << "Base::f(x, c)\n"; }
};

class Derived : public Base {
public:
    void f(int x) { cout << "Derived::f(x)\n"; }
    void f(int x, int y) { cout << "Derived::f(x, y)\n"; }
    void f(double d) { cout << "Derived::f(d)\n"; }
    void test() {
        f(); // Base::f() 호출?, No
        // Compile Error
        f(3, ' '); // Base::f(x, c) 호출? No
        // Derived::f(3, (int)' ') 호출함
        f(3); // OK
        f(3, 32); // OK
        f(5.14); // OK
    };
};

void main() {
    Derived d; d.test();
}
```

```
void f() { cout << "f() called" << endl; } // 전역함수
void f(double d) { cout << "Derived::f(d)\n"; }

class Base {
public:
    void f() { cout << "Base::f()" << endl; }
    void f(int x, char c) { cout << "Base::f(x, c)\n"; }
};

class Derived : public Base {
public:
    void test() {
        f(); // Base::f() 호출

        f(3, ' '); // Base::f(x, c) 호출
        f(3); // Compile Error
        f(3, 32); // OK, Base::f(3, (char)32)
        f(5.14); // 전역함수 호출? No
        // Compile Error
    };
};

void main() {
    Derived d; d.test();
    f();
    f(5.14);
}
```


오버로딩과 함수 재정의, 오버라이딩 비교

25

비교 요소	오버로딩	함수 재정의(가상 함수가 아닌 멤버에 대해)	오버라이딩
정의	매개 변수 타입이나 개수가 다르지만, 이름이 같은 함수들이 중복 작성되는 것	기본 클래스의 멤버 함수를 파생 클래스에서 이름, 매개 변수 타입과 개수, 리턴 타입까지 완벽히 같은 원형으로 재작성하는 것	기본 클래스의 가상 함수를 파생 클래스에서 이름, 매개 변수 타입과 개수, 리턴 타입까지 완벽히 같은 원형으로 재작성하는 것
존재	클래스의 멤버들 사이, 외부 함수들 사이, 그리고 기본 클래스와 파생 클래스 사이에 존재 가능	상속 관계	상속 관계
목적	이름이 같은 여러 개의 함수를 중복 작성하여 사용의 편의성 향상	기본 클래스의 멤버 함수와 별도로 파생 클래스에서 필요하여 재작성	기본 클래스에 구현된 가상 함수를 무시하고, 파생 클래스에서 새로운 기능으로 재작성하고자 함
바인딩	정적 바인딩. 컴파일 시에 중복된 함수들의 호출 구분	정적 바인딩. 컴파일 시에 함수의 호출 구분	동적 바인딩. 실행 시간에 오버라이딩된 함수를 찾아 실행
객체 지향 특성	컴파일 시간 다형성	컴파일 시간 다형성	실행 시간 다형성

가상 함수와 오버라이딩 활용 사례

- 가상 함수를 가진 기본 클래스의 목적
- 가상 함수 오버라이딩
- 동적 바인딩 실행
- 기본 클래스의 포인터 활용

1. 가상 함수를 가진 기본 클래스의 목적

27

- Shape은 상속을 위한 기본 클래스로의 역할
- 가상 함수 **draw()**로 파생 클래스에서 재정의해야 할 함수의 원형(인터페이스)을 보여줌
 - **Shape 객체를 생성할 목적 아님**
 - 각각의 파생 클래스에서 가상 함수 **draw()** 재정의. 자신의 도형을 그리도록 유도

Shape.h

```
class Shape {
    Shape* next;
protected:
    virtual void draw();
public:
    Shape() { next = NULL; }
    virtual ~Shape() {}
    void paint();
    Shape* add(Shape* p);
    Shape* getNext() { return next; }
};
```

Shape.cpp

```
#include <iostream>
#include "Shape.h"
using namespace std;

void Shape::paint() {
    draw();
}

void Shape::draw() {
    cout << "--Shape--" << endl;
}

Shape* Shape::add(Shape *p) {
    this->next = p;
    return p;
}
```

Circle.h

```
class Circle : public Shape {
protected:
    void draw() override;
};
```

```
#include <iostream>
#include "Shape.h"
#include "Circle.h"
using namespace std;
```

```
void Circle::draw() {
    cout << "Circle" << endl;
}
```

Circle.cpp

Rect.h

```
class Rect : public Shape {
protected:
    void draw() override;
};
```

```
#include <iostream>
#include "Shape.h"
#include "Rect.h"
using namespace std;
```

```
void Rect::draw() {
    cout << "Rectangle" << endl;
}
```

Rect.cpp

Line.h

```
class Line : public Shape {
protected:
    void draw() override;
};
```

```
#include <iostream>
#include "Shape.h"
#include "Line.h"
using namespace std;
```

```
void Line::draw() {
    cout << "Line" << endl;
}
```

Line.cpp

2. 가상 함수 오버라이딩

28

- 파생 클래스에서 가상 함수 draw()의 재정의(오버라이딩)
 - ▣ 어떤 경우에도 자신이 만든 draw()가 호출됨을 보장 받음
 - 동적 바인딩에 의해
- 파생 클래스마다 다르게 구현하는 다형성(polymorphism)

```
void Circle::draw() { cout << "Circle" << endl; }
```

```
void Rect::draw() { cout << "Rectangle" << endl; }
```

```
void Line::draw() { cout << "Line" << endl; }
```

- ▣ 함수 이름은 동일하지만 각 파생 클래스별로 가상 함수의 재정의
를 통해 각 도형별로 자신의 도형 모양을 다르게 그릴 수 있게 함

예제 main() 함수

29

```
#include <iostream>
#include "Shape.h"
#include "Circle.h"
#include "Rect.h"
#include "Line.h"
using namespace std;

int main() {
    Shape *pStart= nullptr;
    Shape *pLast;

    pStart = new Circle(); // 처음에 원 도형을 생성한다.
    pLast = pStart;

    pLast = pLast->add(new Rect()); // 사각형 객체 생성
    pLast = pLast->add(new Circle()); // 원 객체 생성
    pLast = pLast->add(new Line()); // 선 객체 생성
    pLast = pLast->add(new Rect()); // 사각형 객체 생성

    // 현재 연결된 모든 도형을 화면에 그린다.
    Shape* p = pStart;
    while(p != nullptr) {
        p->paint();
        p = p->getNext();
    }
```

```
// 현재 연결된 모든 도형을 삭제한다.
p = pStart;
while(p != nullptr) {
    Shape* q = p->getNext(); // 다음 도형 주소 기억
    delete p; // 기본 클래스의 가상 소멸자 호출
    p = q; // 다음 도형 주소를 p에 저장
}
}
```

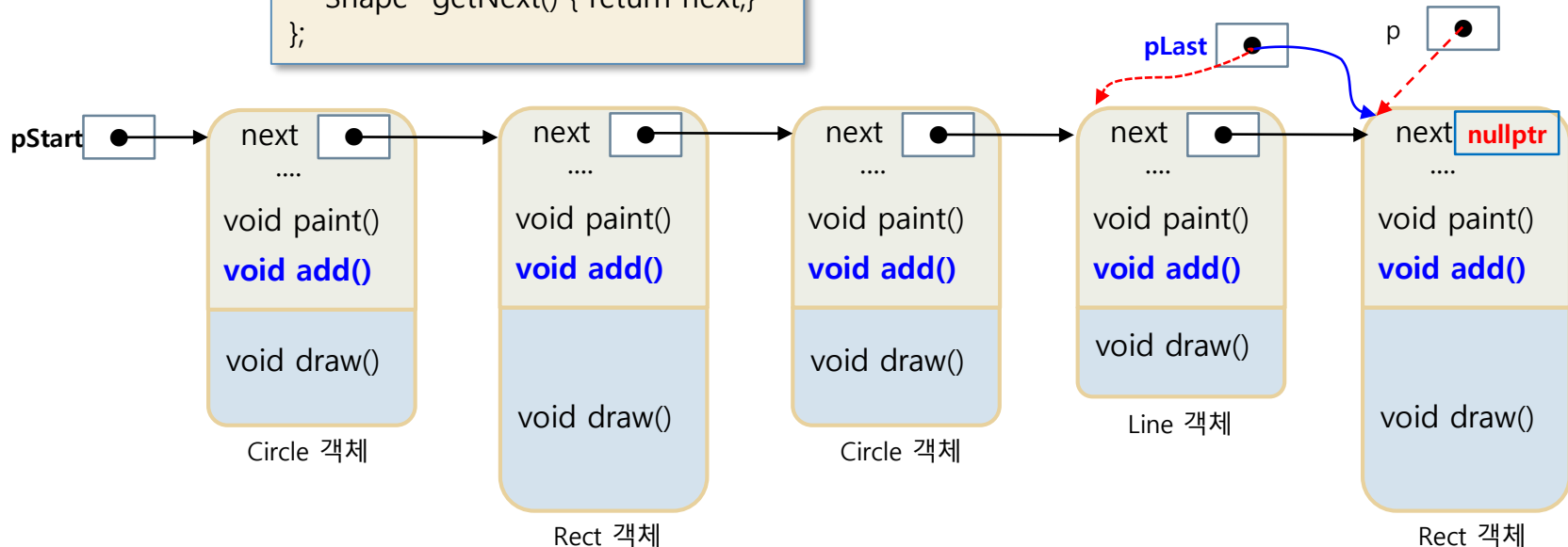
Circle
Rectangle
Circle
Line
Rectangle

main() 함수가 실행될 때 구성된 객체의 연결

30

```
class Shape {  
    Shape* next;  
protected:  
    virtual void draw();  
public:  
    Shape() { next = nullptr; }  
    virtual ~Shape() { }  
    void paint() { draw(); }  
    Shape* add(Shape* p) {  
        this->next = p;  
        return p;  
    }  
    Shape* getNext() { return next; }  
};
```

```
// int main()  
Shape *pStart= nullptr;  
Shape *pLast;  
pStart = new Circle();  
pLast = pStart;  
pLast = pLast->add(new Rect());  
pLast = pLast->add(new Circle());  
pLast = pLast->add(new Line());  
pLast = pLast->add(new Rect());
```



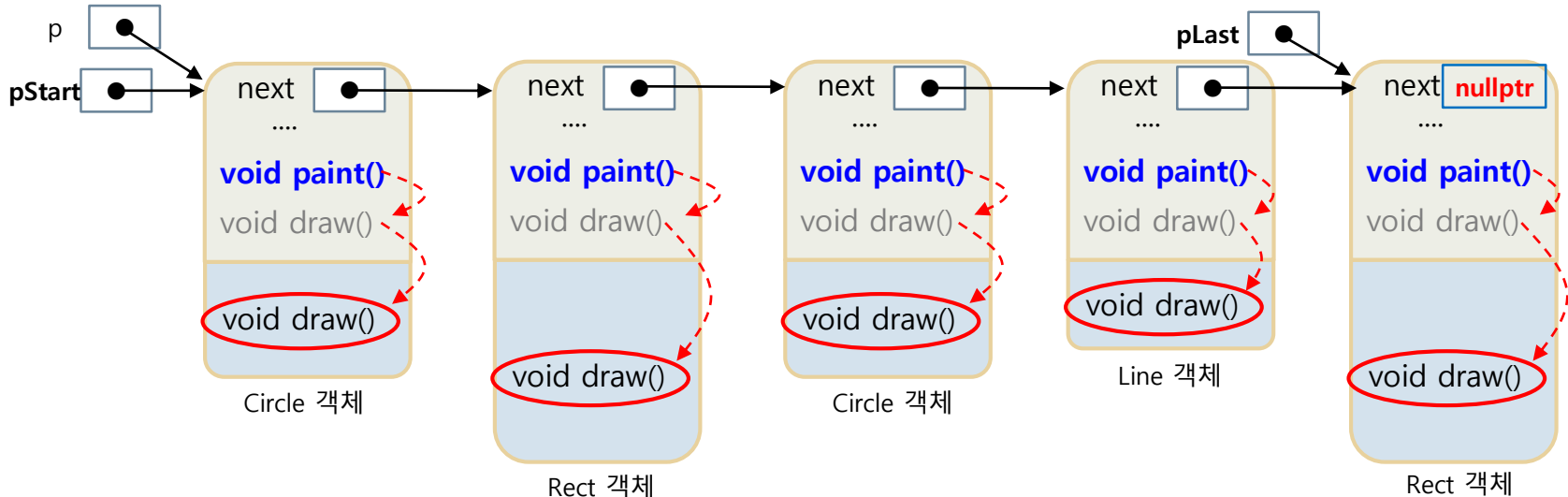
3. 동적 바인딩: 파생 클래스의 가상 함수 실행

31

```
class Shape {
    Shape* next;
protected:
    virtual void draw();
public:
    Shape() { next = nullptr; }
    virtual ~Shape() {}
    void paint() { draw(); }
    Shape* add(Shape* p) {
        this->next = p; return p;
    }
    Shape* getNext() { return next; }
};
```

```
// int main()
// 현재 연결된 모든 도형을 화면에 그린다.
Shape* p = pStart;
while(p != nullptr) {
    p->paint(); // 기본 클래스의 가상함수 draw()를 호출함
    p = p->getNext();
}
/* 위 문장들은 다음과 같이 작성할 수 있음
for (Shape* p = pStart; p != nullptr; p = p->getNext())
    p->paint();
*/
```

- p->paint()의 간단한 호출로 파생 객체에 오버라이딩된 draw() 함수 호출



4. 기본 클래스의 포인터 활용

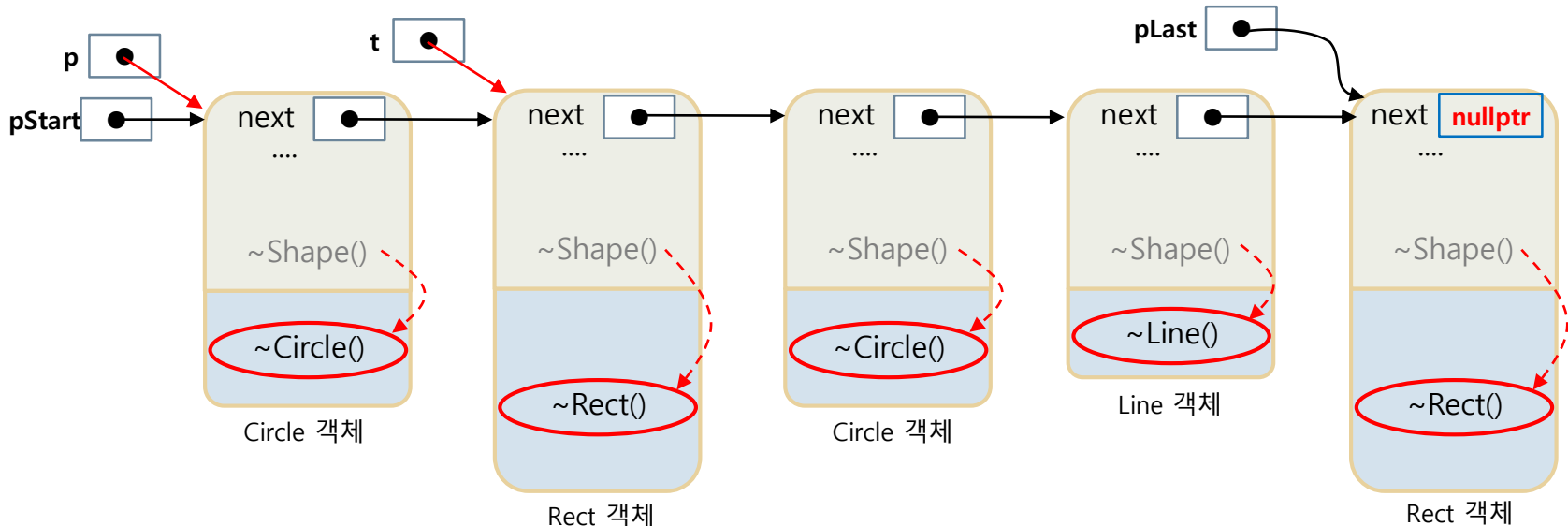
32

- 기본 클래스의 포인터로 파생 클래스 접근
 - ▣ pStart, pLast, p의 타입이 Shape*
 - ▣ 링크드 리스트를 따라 Shape을 상속받은 파생 객체들 접근
 - ▣ p->paint()의 간단한 호출로 파생 객체에 오버라이딩된 draw() 함수 호출

```
class Shape {  
    ...  
public:  
    Shape() { next = nullptr; }  
    virtual ~Shape() { }  
    void paint() { draw(); }  
    ...  
};
```

```
for (Shape* p = pStart; p != nullptr; p = p->getNext())  
    p->paint();
```

```
// 현재 연결된 모든 도형을 삭제한다.  
for (Shape* p = pStart; p != nullptr; p = t) {  
    Shape *t = p->getNext();  
    delete p; // 기본 클래스의 가상 소멸자 실행  
}
```



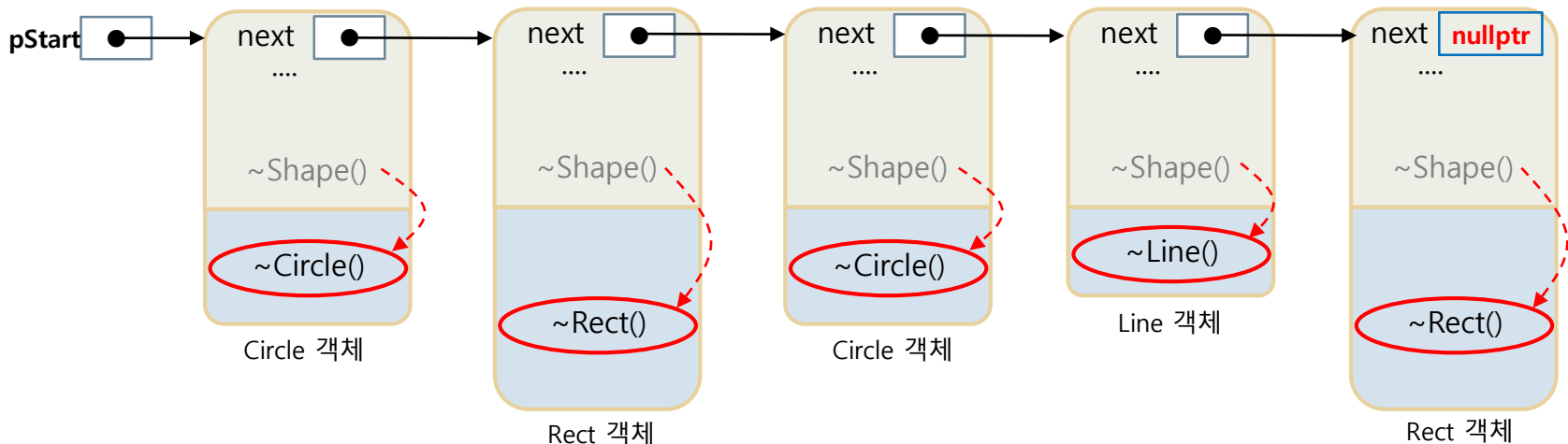
동적 바인딩 및 오버라이딩 기능이 없을 경우

33

```
// 오버라이딩 기능을 활용한 경우  
for (Shape* p = pStart; p != nullptr; p = p->getNext()) {  
    p->paint();  
}
```

업캐스팅된 포인터 p가 실제 어떤 객체인지 조사한 후
원래 객체의 포인터로 다운캐스팅하고,
다운캐스팅된 포인터를 이용하여
오버라이딩된 paint() 함수를 호출함

```
for (Shape* p = pStart; p != nullptr; p = p->getNext()) {  
    if (p가 Line의 객체이면) {  
        Line *line = (Line *)p;  
        line->paint();  
    }  
    else if (p가 Circle의 객체이면) {  
        Circle *circle = (Circle *)p;  
        circle->paint();  
    }  
    else if (p가 Rect의 객체이면) {  
        Rect *rect = (Rect *)p;  
        rect->paint();  
    }  
}
```



순수 가상 함수

34

- 기본 클래스의 가상 함수 목적
 - ▣ 파생 클래스에서 재정의할 함수를 알려주는 역할
 - 실행할 코드를 작성할 목적이 아님
 - ▣ 기본 클래스의 가상 함수를 굳이 구현할 필요가 있을까?
- 순수 가상 함수
 - ▣ pure virtual function
 - ▣ 함수의 **코드가 없고** 선언만 있는 **가상 멤버** 함수
 - ▣ 선언 방법
 - 멤버 함수의 원형 = **0;** 으로 선언

```
class Shape {  
public:  
    virtual void draw() = 0; // 순수 가상 함수 선언  
};
```

추상 클래스

35

- 추상 클래스 : 최소한 하나의 순수 가상 함수를 가진 클래스

```
class Shape { // Shape은 추상 클래스
    Shape *next;
public:
    void paint() {
        draw();
    }
    virtual void draw() = 0; // 순수 가상 함수
};
void Shape::paint() {
    draw(); // 순수 가상 함수라도 호출은 할 수 있다.
}
```

- 추상 클래스의 특징

- 온전한 클래스가 아니므로 객체 생성 불가능

```
Shape shape; // 컴파일 오류
Shape *p = new Shape(); // 컴파일 오류
```

error C2259: 'Shape' : 추상 클래스를
인스턴스화할 수 없습니다.

- 추상 클래스의 포인터는 선언 및 업캐스팅 가능

```
Rect r;
Shape *p = &r;           // 업캐스팅
Shape *p2 = new Rect();  // 업캐스팅
```

추상 클래스의 목적

36

□ 추상 클래스의 목적

- ▣ 추상 클래스의 객체(인스턴스)를 생성할 목적 아님
- ▣ 상속에서 기본 클래스의 역할, 즉 인터페이스 역할을 하기 위함
 - 순수 가상 함수를 통해 파생 클래스에서 구현할 **함수의 형태(원형)**을 보여주는 **인터페이스 역할 (함수의 표준 스펙 정의)**
 - 추상 클래스의 모든 멤버 함수를 순수 가상 함수로 선언할 필요는 없음

추상 클래스의 상속과 구현

37

- 추상 클래스의 상속
 - ▣ 추상 클래스를 단순 상속하면 자동 추상 클래스
- 추상 클래스의 구현
 - ▣ 추상 클래스를 상속받아 순수 가상 함수를 오버라이딩
 - 파생 클래스는 추상 클래스가 아님

Shape은
추상 클래스

Circle도
추상 클래스

순수 가상 함수인
draw()를 구현하
지 않았기 때문에

```
class Shape {  
public:  
    virtual void draw() = 0;  
};  
  
class Circle : public Shape {  
public:  
    string toString() { return "Circle 객체"; }  
};
```

Shape shape; // 객체 생성 오류
Circle waffle; // 객체 생성 오류

추상 클래스의 단순 상속



Shape은
추상 클래스

Circle은
추상 클래스 아님

순수 가상 함수 구
현(오버라이딩)

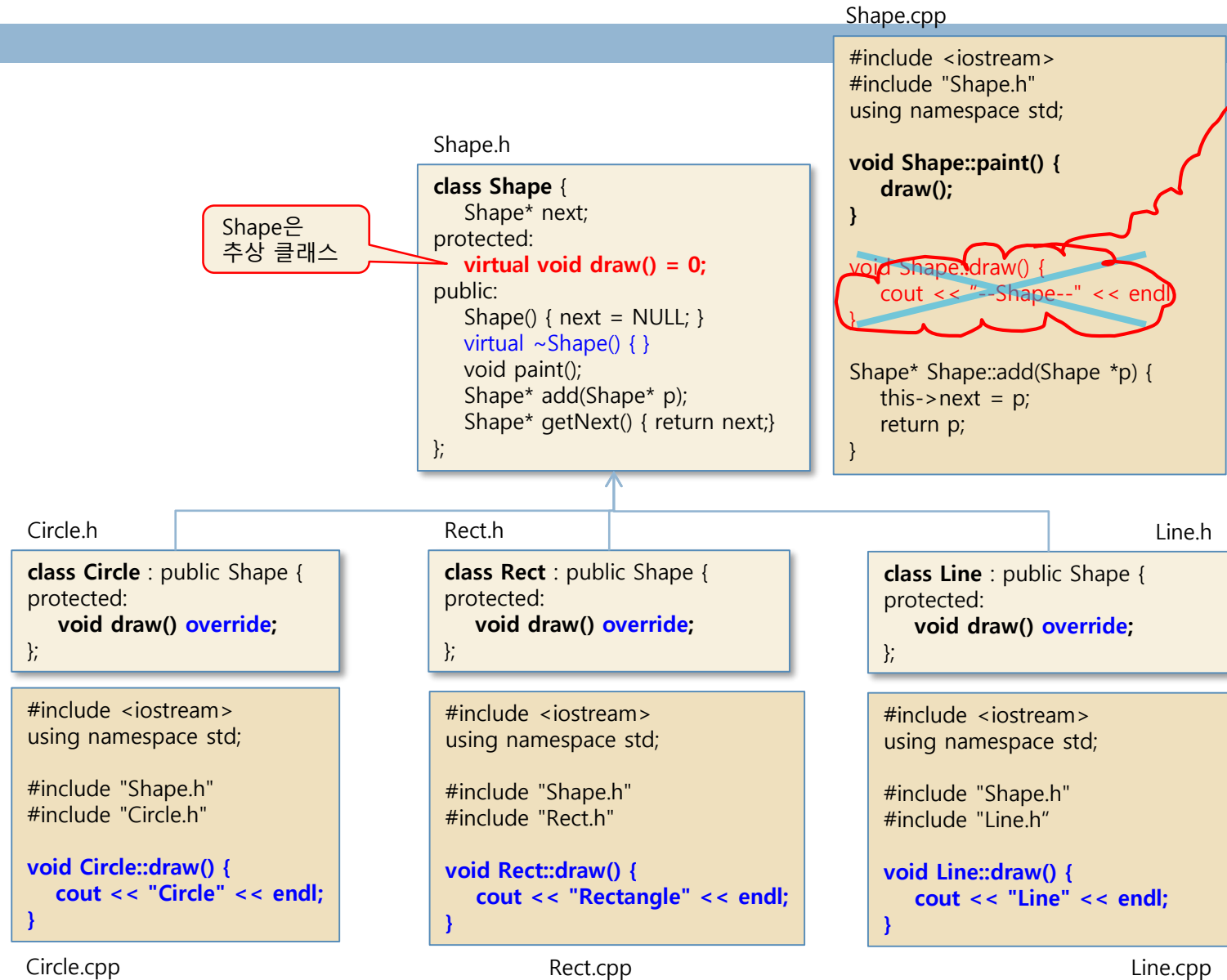
```
class Shape {  
public:  
    virtual void draw() = 0;  
};  
  
class Circle : public Shape {  
public:  
    void draw() override {  
        cout << "Circle";  
    }  
    string toString() { return "Circle 객체"; }  
};
```

Shape shape; // 객체 생성 오류
Circle waffle; // 정상적인 객체 생성

추상 클래스의 구현

Shape을 추상 클래스로 수정

38



예제 9-6(실습) 추상 클래스 구현 연습 1

39

다음 추상 클래스 Calculator를 상속받아 GoodCalc 클래스를 구현하라.

```
class Calculator {
public:
    virtual int add(int a, int b) = 0; // 두 정수의 합 리턴
    virtual int subtract(int a, int b) = 0; // 두 정수의 차 리턴
    virtual double average(int a [], int size) = 0; // 배열 a의 평균 리턴. size는 배열의 크기
};
```

```
#include <iostream>
using namespace std;

// 이 곳에 Calculator 클래스 코드 필요
```

```
class GoodCalc : public Calculator {
public:
    int add(int a, int b) override { return a + b; }
    int subtract(int a, int b) override { return a - b; }
    double average(int a [], int size) override {
        double sum = 0;
        for(int i=0; i<size; i++)
            sum += a[i];
        return sum/size;
    }
};
```

순수 가상 함수 구현

```
int main() {
    int a[] = {1,2,3,4,5};
    Calculator *p = new GoodCalc();
    cout << p->add(2, 3) << endl;
    cout << p->subtract(2, 3) << endl;
    cout << p->average(a, 5) << endl;
    delete p;
}
```

5
-1
3

추상 클래스 구현 연습 2

40

```
class Calculator {  
public:  
    virtual int add(int a, int b) = 0; // 두 정수의 합 리턴  
    virtual int subtract(int a, int b) = 0; // 두 정수의 차 리턴  
    virtual double average(int a [], int size) = 0; // 배열 a의 평균 리턴. size는 배열의 크기  
};
```

```
#include <iostream>  
using namespace std;  
  
// 이 곳에 앞 페이지의 Calculator, GoodCalc 클래스 코드 필요  
  
class BadCalc : public Calculator {  
public:  
    int add(int a, int b) override { return (2*a + 2*b)/2; }  
    int subtract(int a, int b) override { return 2*a/2 - b*b/b; }  
    double average(int a [], int size) override {  
        double sum = 0;  
        for(int i=0; i<size; i++)  
            sum += (a[i]+a[i]-a[i]) * 10 / 10;  
        return sum/size;  
    }  
};
```

```
void calculate(Calculator *p) {  
    int a[] = {1,2,3,4,5};  
    cout << p->add(2, 3) << endl;  
    cout << p->subtract(2, 3) << endl;  
    cout << p->average(a, 5) << endl;  
    delete p;  
}  
  
int main() {  
    calculate(new GoodCalc());  
    calculate(new BadCalc());  
}
```

```
5  
-1  
3  
5  
-1  
3
```


예제 9-7(실습) 추상 클래스를 상속받는 파생 클래스 구현 연습

41

다음 코드와 실행 결과를 참고하여 추상 클래스 Calculator를 상속받는 Adder와 Subtractor 클래스를 구현하라.

```
#include <iostream>
using namespace std;

class Calculator {
    void input() {
        cout << "정수 2 개를 입력하세요>> ";
        cin >> a >> b;
    }
protected:
    int a, b;
    virtual int calc(int a, int b) = 0; // 두 정수의 합 리턴
public:
    void run() {
        input();
        cout << "계산된 값은 " << calc(a, b) << endl;
    }
};

int main() {
    Adder adder;
    Subtractor subtractor;
    adder.run();
    subtractor.run();
}
```

adder.run()에 의한 실행 결과

subtractor.run()에 의한 실행 결과

```
정수 2 개를 입력하세요>> 5 3
계산된 값은 8
정수 2 개를 입력하세요>> 5 3
계산된 값은 2
```

예제 9-7 정답

42

```
#include <iostream>
using namespace std;

class Calculator {
    void input() {
        cout << "정수 2 개를 입력하세요>> ";
        cin >> a >> b;
    }
protected:
    int a, b;
    virtual int calc(int a, int b) = 0; // 두 정수의 합 리턴
public:
    void run() {
        input();
        cout << "계산된 값은 " << calc(a, b) << endl;
    }
};

int main() {
    Adder adder;
    Subtractor subtractor;
    adder.run();
    subtractor.run();
}
```

정수 2 개를 입력하세요>> 5 3
계산된 값은 8
정수 2 개를 입력하세요>> 5 3
계산된 값은 2

```
class Adder : public Calculator {
protected:
    // 순수 가상 함수 구현
    int calc(int a, int b) override {
        return a + b;
    }
};

class Subtractor : public Calculator {
protected:
    // 순수 가상 함수 구현
    int calc(int a, int b) override {
        return a - b;
    }
};
```